

Implement a Prototype of an Accessible Sortable Table

GSoC Project proposal for Kolibri

PERSONAL DETAILS & CONTACT INFORMATION :

- **Full Name:** Anoushka Jha
- **University:**
- **Date of Enrollment:**
- **Expected Graduation date:**
- **Degree:** Bachelor of Technology
- **Email Address:**
- **Github Username:** <https://github.com/BabyElias>
- **Phone Number:**

TABLE OF CONTENTS

S.NO	NAME
1.	<u>SYNOPSIS</u>
2.	<u>BENEFITS TO THE COMMUNITY</u>
3.	<u>EXPECTED RESULTS</u>
4.	<u>ACCESSIBILITY</u>
5.	<u>APPROACH / IMPLEMENTATION</u>
6.	<u>TIMELINE & DELIVERABLES</u>
7.	<u>ADDITIONAL INFORMATION</u>

SYNOPSIS

The Kolibri Design System contains resources for designers and developers building Kolibri products. With inclusivity at the heart of Learning Equality's mission, the purpose of KDS is to reduce the time it takes to design & build products while ensuring accessibility, responsiveness, and cross-browser compatibility.

Currently, Data tables in Kolibri and Studio are implemented using CoreTable and SortableHeader components. However, this implementation does not support adding sorting functionality to the tables, which is one of the most common tools for data management. Sorting helps arrange data meaningfully to make it easier to understand, compare, and visualize. Also, using these components leaves too much to the developer to ensure accessibility across all user sections. This project aims to develop a new KDS component, a sortable table to provide a solution to these problems.

BENEFITS TO THE COMMUNITY

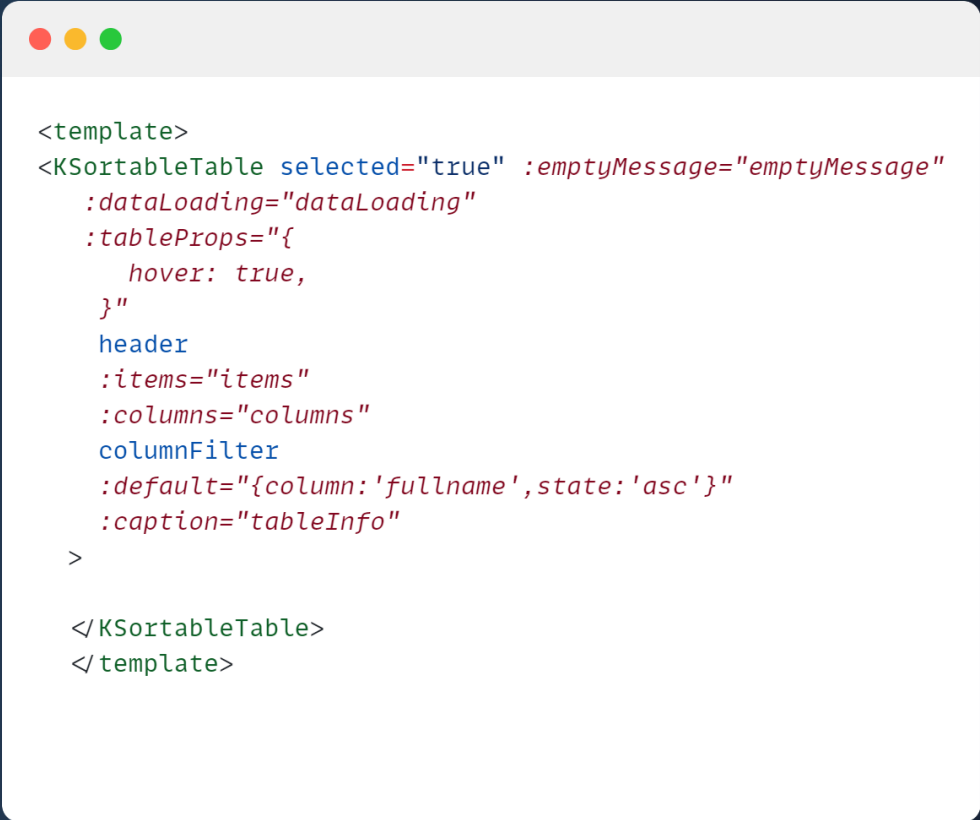
The purpose of this project is to create a new KDS component, **KTable**.

The features of this component will be -

- Provide an easy-to-use, standard sortable experience
- Meet best practices for screen reader accessibility
- Ensure best-practice keyboard navigation throughout
- Support internationalization
- Have easy reusability across all Kolibri and KDP products

EXPECTED RESULTS

This is a sample code representation of the expected final, ready-to-use component.



```
<template>
<KSortableTable selected="true" :emptyMessage="emptyMessage"
  :dataLoading="dataLoading"
  :tableProps="{
    hover: true,
  }"
  header
  :items="items"
  :columns="columns"
  columnFilter
  :default="{column: 'fullname', state: 'asc'}"
  :caption="tableInfo"
>

</KSortableTable>
</template>
```

where,

Columns - corresponds to the array of objects containing name, custom styling, and sorting options

Items - corresponds to the array of objects containing information about data to be filled in individual rows

NOTE: *There might be more components between the KSortableTable opening and closing tags, which are not shown in the image above.*

```
columns: [  
  {  
    key: 'avatar',  
    ariaLabel: 'avatar',  
    filter: false,  
    sorter: false,  
  },  
  {  
    key: 'Full Name',  
    _style: { width: '20%' },  
  },  
  {  
    key: 'Birth Year',  
    _style: { width: '20%' }  
  },  
  {  
    key: 'Date Added',  
    _style: { width: '20%' }  
  },  
  {  
    key: 'Progress',  
    label: '',  
    _style: { width: '1%' },  
    filter: false,  
    sorter: false,  
  },  
],
```

Representation of Data in columns

```
items: [
  {
    id: 1,
    name: 'User 1',
    avatar: '1.jpg',
    username: 'user1',
    birth: '2004'
    dateAdded: '2022/01/01',
    progress: '100',
  }
]
```

Representation of Data in rows (Items)

Users					
	Full Name ▼	UserName ⇅	Birth Year ⇅	Date Added ⇅	Progress ⇅
☐			2004	X Days ago	100%
☐					75%
☐					
☐					

Sample design of the table we aim to create with this code.

ACCESSIBILITY ACCEPTANCE CRITERIA

- *FOR USERS WHO USE SCREEN-READERS*

- Identifies itself as a table when perceived by the screen reader
- Has a caption or heading to define its purpose
- Column Headers & Row Headers are identified with screen reader shortcuts
- Audible indications or feedback when navigating and selecting rows
- Ability to easily identify and modify content in editable cells
- The table design should be as simple as possible, preferably displayed on a single page.
- Consistent and easy keyboard navigation throughout the table
 - Have table as tab-focussed to ensure that table can be navigated using keyboard
 - Ability to navigate from cell-to-cell easily using keyboard shortcuts in a defined manner
 - If rows are selectable, ability to select rows using keyboard
 - If cells are editable, ability to edit cells and save changes using keyboard

- *FOR USERS WITH OTHER DISORDERS (SPECIALLY , COGNITIVE & MOTOR DISORDERS)*

- Maximum signal(relevant info) to noise(irrelevant info) ratio
- Minimalistic color palette
- More whitespace for more readability
 - Less borders, well spaced content to distinguish between cells
- Highlighted active cells for easier understanding of data
- Consistent formatting throughout the table for predictability
- Manageable amount of information in each cell to avoid overwhelming cognitive load

HOW CAN THESE BE IMPLEMENTED?

- The table would be semantically marked up using appropriate HTML elements (<table>, <thead>, <tbody>, <th>, <tr>, <td>). This structure provides a clear hierarchy and context for screen reader users.
- Using **scope='col'/row'** to identify row/column header for screen readers.
- The **aria-selected** attribute will be used convey the selected state of rows. Additionally, the **aria-label** attribute on the table would provide a concise and informative label to give an overview of the table's content.
- The **aria-sort** attribute will be used to convey the sorted state/order of columns/rows.
- In case of editable tables, the **aria-readonly** attribute can be added to the non-editable fields.

HTML VIEW OF THE EXPECTED TABLE LAYOUT

```
<table class="sortable">
  <caption>
    Information about students enrolled
    <span class="sr-only">, column headers with buttons are sortable.</span>
  </caption>
  <thead>
    <tr>
      <th scope="col" aria-sort="ascending">
        <button>First Name</button>
      </th>
      <th scope="col">
        <button>Last Name</button>
      </th>
      <th>
        <button>UserName</button>
      </th>
      <th scope="col" class="no-sort">Progress Bar</th>
      <th scope="col" class="date">
        <button>Date added</button>
      </th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Anoushka</td>
      <td>Jha</td>
      <td>ajha_6</td>
      <td aria-label="Student's learning progress"><Progress /></td>
      <td class="date">7 days ago</td>
    </tr>
  </tbody>
</table>
```

A BETTER EXPLAINED VERSION OF THE EXPECTED HTML TABLE

```

<table class="sortable">
  <caption>
    Information about students enrolled
    <span class="sr-only">, column headers with buttons are sortable.</span>
  </caption>
  <thead>
    <tr>
      <th scope="col" aria-sort="ascending">
        <button>First Name</button>
      </th>
      <th scope="col">
        <button>Last Name</button>
      </th>
      <th>
        <button>User Name</button>
      </th>
      <th scope="col" class="no-sort">Progress Bar</th>
      <th scope="col" class="date">
        <button>Date added</button>
      </th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Anoushka</td>
      <td>Jha</td>
      <td>ajha_6</td>
      <td aria-label="Student's learning progress"><Progress/></td>
      <td class="date">7 days ago</td>
    </tr>
  </tbody>
</table>

```

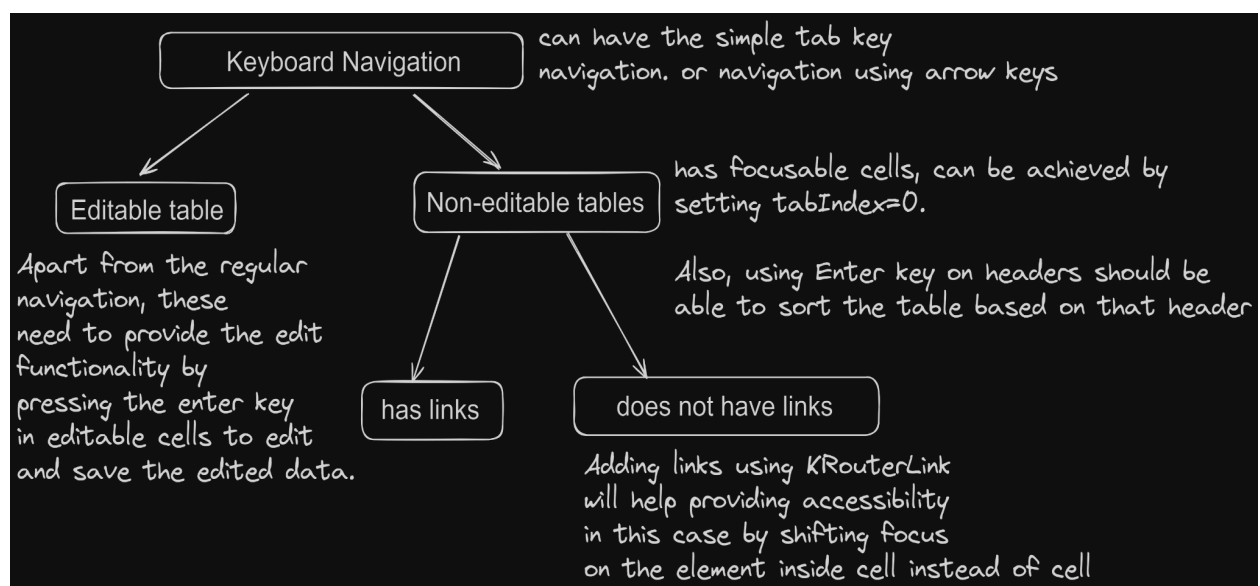
→ sr-only -> for information visible to only screen readers

→ scope=col to identify column header
aria-sort=ascending to identify state of sorting

→ class="no-sort" to signify that this cannot be sorted

→ Since this is a non-conventional data type, so separate information about this should be conveyed using aria-label

• KEYBOARD NAVIGATION :



TESTING ACCESSIBILITY

- **Screen Reader Compatibility Testing :**
 - Using [NVDA Screen Reader](#) : I gained slight hands-on experience in working with this screen reader while researching accessibility standards for this project and testing it on some real world accessible tables.
- **Keyboard Navigation Testing :**
 - Tab Navigation: Testing the website's tab order by navigating through all interactive elements (links, buttons, form fields) using only the keyboard's Tab key. Ensuring that the focus moves in a logical order and that all interactive elements are reachable.
 - Focus Styles: Verifying that interactive elements have visible focus states. This is crucial for users navigating with a keyboard to know which element they are currently focused on.

RELEVANT ARTICLES/LINKS

- <https://www.w3.org/WAI/ARIA/apg/patterns/table/>
- <https://www.w3.org/WAI/ARIA/apg/patterns/grid/>
- <https://www.w3.org/WAI/ARIA/apg/practices/keyboard-interface/>
- <https://inclusive-components.design/data-tables/>
- <https://adrianroselli.com/2021/04/sortable-table-columns.html>
- https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/table_role
- <https://developer.mozilla.org/en-US/docs/Learn/HTML/Tables/Advanced>
- <https://www.smashingmagazine.com/2022/12/accessible-front-end-patterns-responsive-tables-part1/>

REFERENCES FROM PREVIOUS ISSUES :

- <https://github.com/learningequality/kolibri-design-system/issues/328>
- <https://github.com/learningequality/kolibri-design-system/issues/330>
- <https://github.com/learningequality/kolibri-design-system/issues/332>
- <https://github.com/learningequality/kolibri/issues/4620>
- <https://github.com/learningequality/kolibri/issues/7999>

APPROACH

In order to implement KSortable table with proper accessibility and internationalization standards, I plan to move forward in a phase-wise approach wherein we first ensure the accessibility and responsiveness of the basic component and then move forward with internationalization and further enhancements.

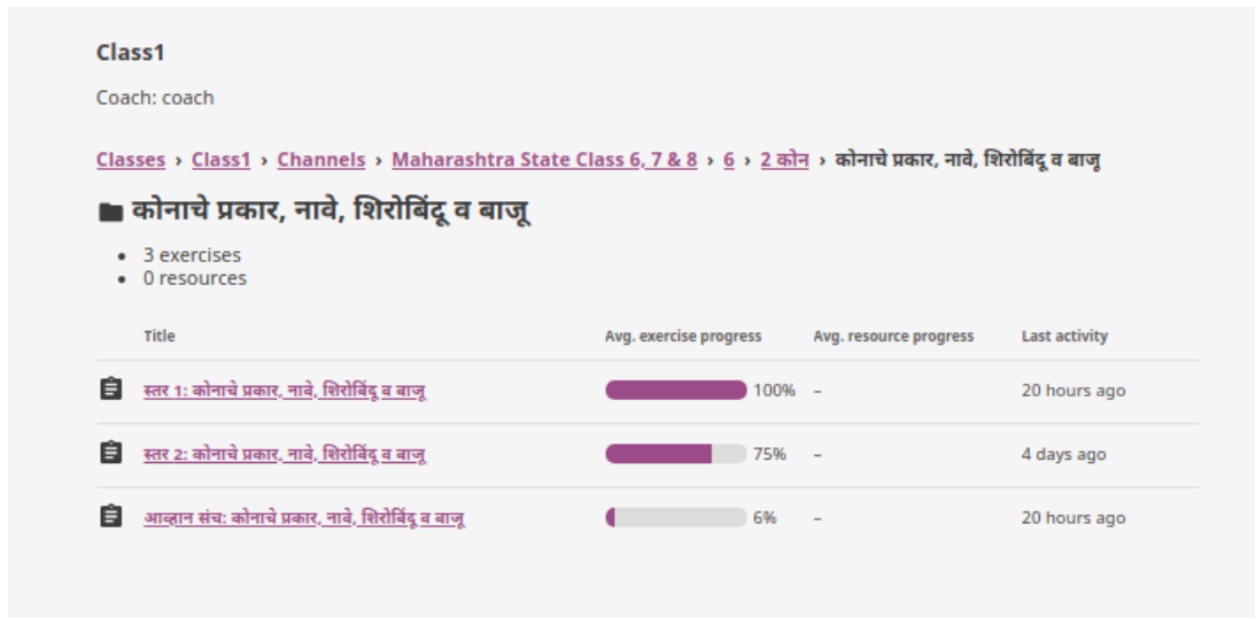
PHASE I - ACCESSIBILITY & RESPONSIVENESS

STEP 1 :

- ❖ Determine use cases for sortable tables
- ❖ Determine all possible data types, their alignment in a table cell
- ❖ their sorting criteria (and whether, they should be considered for sorting or not), &
- ❖ suitable aria-labels for screen-reader accessibility for these different data types.

Some possible use cases of these tables (in Kolibri and Studio) can be seen as :

- Coach Reports - Lessons



The screenshot displays a web interface for 'Class1' with a coach named 'coach'. It shows a breadcrumb trail: 'Classes > Class1 > Channels > Maharashtra State Class 6, 7 & 8 > 6 > 2 कोन > कोनाचे प्रकार, नावे, शिरोबिंदू व बाजू'. Below this, a section titled 'कोनाचे प्रकार, नावे, शिरोबिंदू व बाजू' lists '3 exercises' and '0 resources'. A table follows with columns: 'Title', 'Avg. exercise progress', 'Avg. resource progress', and 'Last activity'. The table contains three rows of lesson data, each with a progress bar and a timestamp.

Title	Avg. exercise progress	Avg. resource progress	Last activity
स्तर 1: कोनाचे प्रकार, नावे, शिरोबिंदू व बाजू	100%	-	20 hours ago
स्तर 2: कोनाचे प्रकार, नावे, शिरोबिंदू व बाजू	75%	-	4 days ago
आव्हान संच: कोनाचे प्रकार, नावे, शिरोबिंदू व बाजू	6%	-	20 hours ago

- Coach Reports - Learners

Learners						
LESSONS	QUIZZES	GROUPS	LEARNERS			
Name	Groups	Exercises completed	Resources viewed	Average quiz score	Last activity	 
 Carine	Blue group	3	5	45%	2 hours ago	
 David	Green group	4	9	59%	5 hours ago	
 Ivan	Red group	2	3	65%	1 month ago	
 Jamie	Green group	5	8	80%	2 years ago	

- Administration (Channels)

Kolibri Studio

Admin

CHANNELS

USERS

4 channels

Channel Type

Live

Language

Search for a channel...

Search for channels by their names, tokens, ids, or editors

☐ Channel name ↑		Editors	Viewers	Date created	Last updated	Demo URL	Source URL	Actions	
☐ Empty Channel	☒	0 B	2 ☒	1	December 29, 2023	2 months ago	//hh ☒	☒ Not set ☒	ACTIONS ▾
☐ Imported Channel	☒	2 B	1 ☒	0	December 29, 2023	2 months ago	Not set ☒	☒ Not set ☒	ACTIONS ▾
☐ Published Channel ☒	☒	KB	1 ☒	0	December 29, 2023	2 days ago	Not set ☒	☒ Not set ☒	ACTIONS ▾
☐ Ricecooker Channel	☒	0 B	2 ☒	1	December 29, 2023	2 months ago	Not set ☒	☒ Not set ☒	ACTIONS ▾

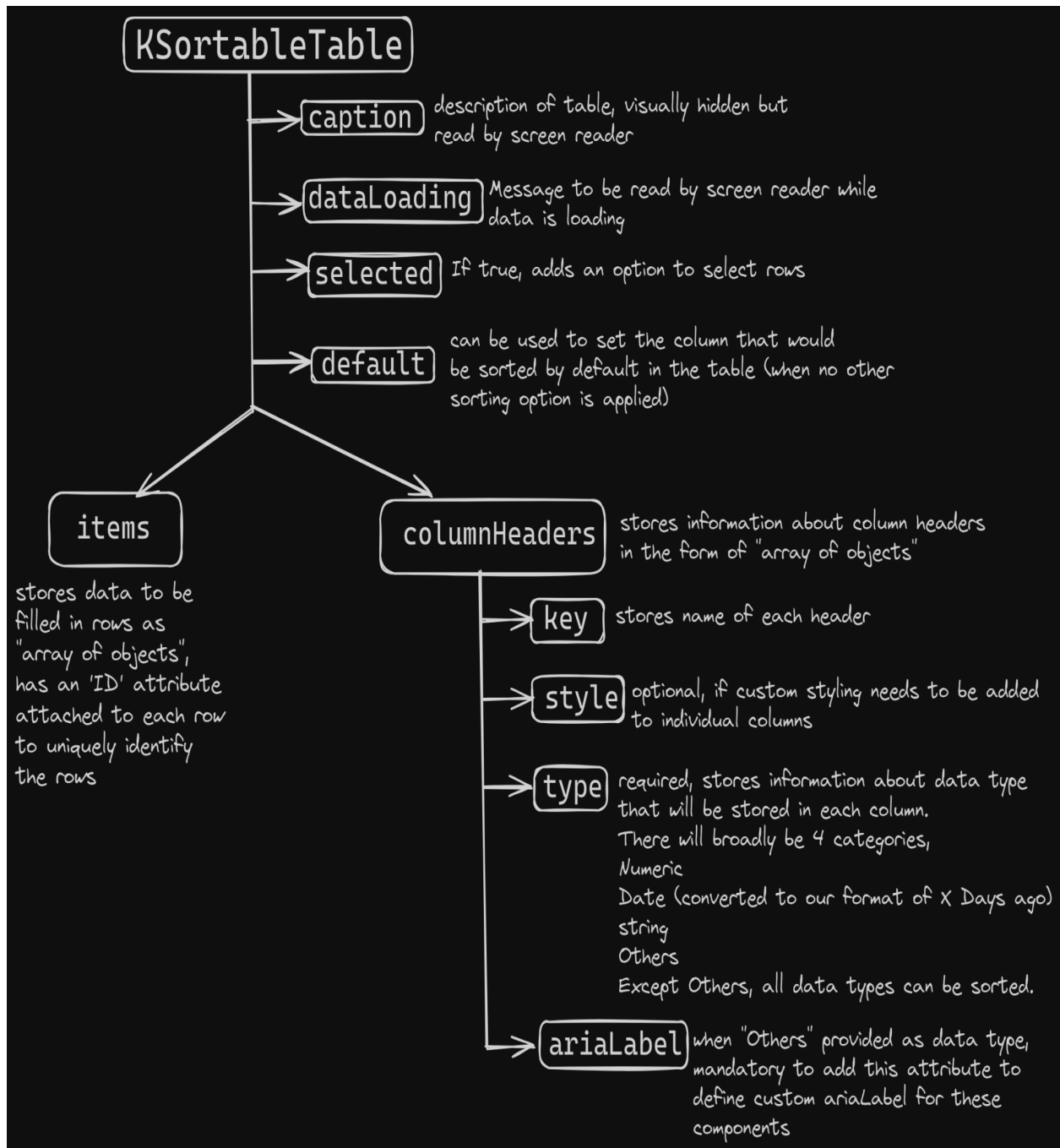
Rows per page:

25 ▾

1-4 of 4

STEP 2:

- ❖ Based on Step 1, implement KSortableTable component for a *basic "column header"* only table that supports single column sorting as :



Sorting functionality will be implemented differently for all possible datatypes based on their requirements.

STEP 3:

- ❖ Use only semantic HTML elements for table implementation to ensure accessibility.
- ❖ Add aria-labelling for screen-reader accessibility to this table, as :
 - Use **scope="col"** with each th tag
 - Add **aria-sort** labels, that will be toggled by **aria-pressed** actions
 - Add separate aria-label features for custom data types (described by *Others*)
- ❖ Add Responsiveness to the table by creating scroll bars for movement. Here, we can have the traditional scroll movement OR
 - Make the first column sticky and others scrollable, if row-wise identification of information by values stored in the first column is important and the table is lengthy. For eg : A Students Table with first name, last name & marks in all subjects. Here, first name - last name column can be made sticky while others are scrollable.

PHASE 2 - INTERNATIONALIZATION & FURTHER ENHANCEMENTS

STEP 4:

- ❖ Add internationalization standards to the KSortableTable component.
 - Identify standard text on the website that needs to be translated, including labels, buttons, error messages, and other user interface elements.
 - Separate text from code by using placeholders or keys for the text that needs to be translated.
 - Define messages in the **<script>** section as attributes of the component definition.
- ❖ Use RTL property wherever needed.

STEP 5:

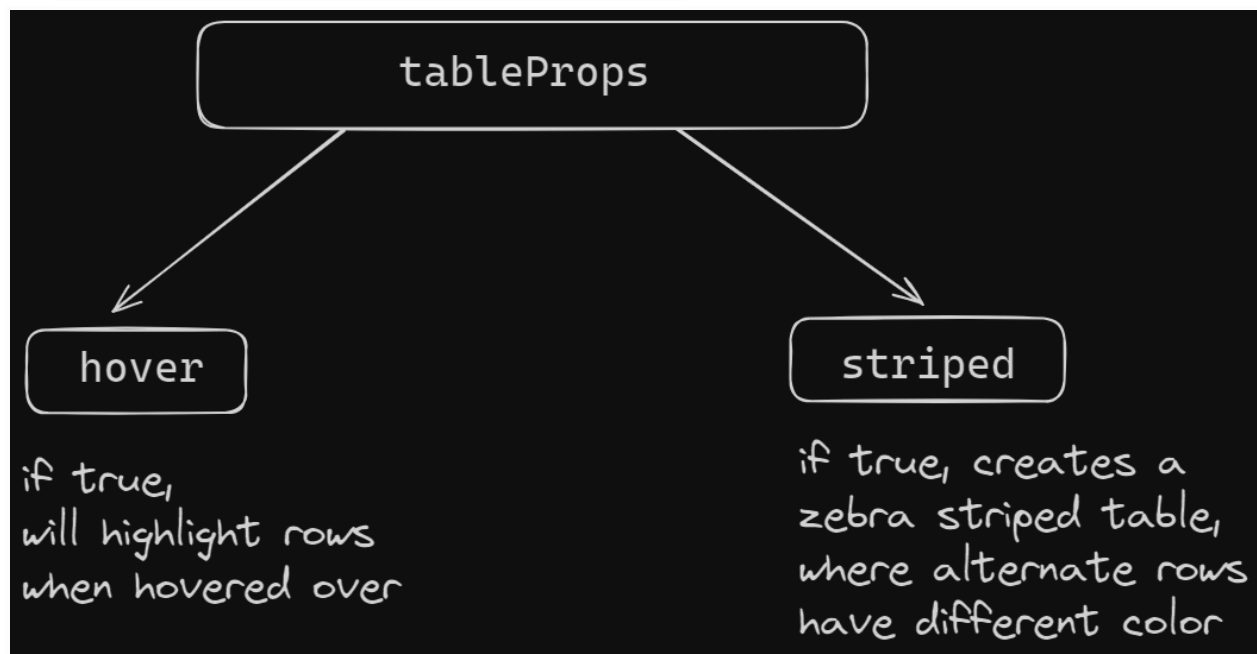
- ❖ Add support for table with both row & column headers.
- ❖ Enhance screen reader accessibility by defining aria-labeling as both **id** & **headers** for each td element.
- ❖ Implement Sorting functionality for table based on row headers. Currently, this would be implemented only for tables where all columns have same datatype.

STEP 6:

- ❖ Add support for editable fields with proper accessibility.
- ❖ The table will support in-place editing, where data can be edited directly into the table without need of any additional forms or dialog boxes. It is similar to the kind of data editing that is present in most Kolibri tables currently.

STEP 7:

- ❖ Add smooth animation for table sorting, to enhance visual appeal.
- ❖ Add options for custom UI to the table as :



EXTENDED GOALS:

- ❖ Configure Table filter options.
 - Feature to view only selected columns at a time
 - Feature to find data related to specific keywords or type
- ❖ Configure Pagination, to support large data tables.
 - Accessibility is generally a concern for Pagination. However, if needed, proper accessibility criterias can be set for implementation.
- ❖ Based on [SLACK Discussion](#)
 - Implementation of sorting according to vertical headers where columns have different datatypes
 - This would be more of re-ordering rather than sorting and can be partially implemented by assigning a custom sort-key to each attribute header.

TIMELINE & DELIVERABLES

	PERIOD	TASK
MILESTONE 1	After proposal submission [April 2 - May 26]	• Research all table use cases and determine a one-fit component for all. • Also, read more about accessibility and internationalization standards suited to our needs. (Step 1) • Try out basic implementation methods for KSortableTable
	Week 1 and 2 [May 29 - June 10]	• Implement KSortableTable for column header only tables. (Step 2)
	Week 3 and 4 [June 11 - June 25]	• Add aria-labelling and responsiveness to the component. (Step 3)
MILESTONE 2	Week 5 and 6 [June 26 - July 8]	BUFFER
	Week 7 and 8 [July 9 - July 23]	• Add internationalization standards.(Step 4)
	Week 9 and 10 [July 24 - Aug 7]	• Add support for table with both row & column headers. (Step 5)
	Week 11 and 12 [Aug 8 - Aug 22]	• Add support for editable fields. • Add options for custom UI. (Step 6 & Step 7)

ADDITIONAL INFORMATION

About Me :



Hey!

This is Anoushka Jha (or [BabyElias](#) ><), a pre-final yearite from the Indian Institute Of Technology (BHU), Varanasi, India. I am a frontend web developer proficient in HTML, CSS, JavaScript & VueJS along with experience in working with ReactJS, NextJS & Nuxt. I love exploring new technologies & contributing to the Open-Source community, making a positive difference with my code.

Apart from this, I enjoy dancing and listening to music in my free time :)

Skills :

Skill Name	Proficiency (1-5)
HTML	4
CSS	4.5
VueJS	4.5
Git/Github	4.5
ReactJS/NextJS/Nuxt	3.5

Previous Contributions to Learning Equality :

ISSUE TITLE	LINK TO ISSUE AND/OR PR	ADDITIONAL NOTES
KRouterLink to Studio: Replace <router-link> in Studio with <KRouterLink>	Issue PR 1 PR 2 PR 3	[Merged]
KRadioButton redundant v-model/value	Issue PR	[Merged]
Remove 3rd party GH action from the changelog workflow	Issue PR	[Merged]
Refactor Icon to use KIcon instead of VIcon	Issue	[Work in Progress]
Broken "develop" branch link	Issue PR	[Merged]
The yarn dev-only misspelled in 01_getting_started.md	Issue PR	[Merged]

Why Learning Equality ?

The equity-driven values. The idea that drives Learning Equality. The work culture, the sections of the community they target, and the entire ideology that runs this organisation.

My idea of technology has always been to put it to use where it can positively impact the lives of people who really need it. And Learning Equality truly aligns with all of it. Throughout these few months that I have worked with Learning Equality, it has been one of my most cherishable Open-Source contribution experiences so far. The entire community, from maintainers to fellow contributors, is all so welcoming and willing to help. It makes me happy to be part of a team that shares equal enthusiasm and motivation towards creating a more equitable society as me :)

Plans after GSoC :

GSoC or not, I would love to continue contributing to Learning Equality and give my little contribution to anything that I can do to help bridge the digital gap that exists around us.

Other Commitments:

I do not have any other plans and commitments this summer except GSoC. I'll have my mid-semester exams somewhere in the month of July, the dates of which have not been announced yet. However, I have allotted a week of buffer period at half the GSoC coding period to meet these commitments.

THANK YOU!